Quantum Query Complexity of the Maximum of a Totally Monotone Matrix

Sebastian Senge
Independent Researcher

Abstract

We study the number of quantum queries needed to find the largest entry of a totally monotone $r \times c$ matrix, where a query returns one entry. Write $n = \max(r, c)$. Classically $\Theta(n)$ queries are necessary and sufficient. Our results are these:

- Deciding whether the maximum is at least a given value takes $O(\sqrt{n} \log n)$ queries.
- The maximum and its position take $O(\sqrt{n} \log^3 n \log\log n)$ queries, with no dependence on the range of the entries.
- An explicit nested quantum search takes $\sqrt{n} \cdot 2^{O(\sqrt{\log n \log\log n})}$ queries, and $\tilde{O}(n^{3/4})$ queries in its simplest form.
- At least $\Omega(\sqrt{n})$ queries are necessary, already for matrices satisfying the reverse quadrangle inequality.
- The diameter of a strictly convex polygon with n vertices takes $\tilde{O}(\sqrt{n})$ queries, and $\Omega(\sqrt{n})$ queries are necessary.

These are query bounds on a noiseless model. The two sharpest upper bounds go through the adversary bound and give no bound on gates or time.

1 Introduction

A matrix M with entries in a totally ordered set is *totally monotone* if for all rows $i < i'$ and columns $j < j'$,

$$M[i, j] < M[i, j'] \implies M[i', j] < M[i', j'].$$

This class contains every matrix with the *reverse quadrangle inequality* $M[i, j]+M[i', j'] \geq M[i, j']+M[i', j]$. Such matrices are a standard tool in computational geometry and dynamic programming, and the algorithm of [1] finds all row maxima of an $r \times c$ totally monotone matrix with $O(r + c)$ entry reads.

Here a query (i, j) returns $M[i, j]$, and the task is to find the largest entry of M. Quantum maximum finding [5] over all entries costs $\Theta(\sqrt{rc})$ queries, which is $\Theta(n)$ on square matrices, so a quantum gain has to use the structure. We show that the structure gives a quadratic gain.

To our knowledge there is no earlier quantum result on totally monotone matrix search, or on the diameter of a convex polygon. The closest result known to us is the bound of [8] for a general point set, on which the structure used here is not available.

Techniques. Computing the row maximum of a middle row splits a matrix into two parts with windows of columns whose widths sum to $c + 1$. A direct nesting of quantum searches pays a polylogarithmic overhead at every level of the recursion, which leaves a subpolynomial factor. In Section 5 we run the same recursion on filtered factorization norms [7], which compose with no such loss, and we obtain two composition lemmas for this purpose.

Problem	Previous best bound	Our bound	Lower bound	Section
Is $\max M \geq \theta$?	$O(n)$ [1]	$O(\sqrt{n}\log n)$	$\Omega(\sqrt{n})$	5
$\max M$ and its position	$O(n)$ [1]	$O(\sqrt{n}\log^3 n \log\log n)$	$\Omega(\sqrt{n})$	5
Same, explicit search	$O(n)$ [1]	$\sqrt{n}\cdot 2^{O(\sqrt{\log n \log\log n})}$	$\Omega(\sqrt{n})$	4
Polygon diameter, decision	$\tilde{O}(n^{3/4})$ [8]	$O(\sqrt{n}\log^{3/2} n)$	$\Omega(\sqrt{n})$	6
Polygon diameter, value	$\tilde{O}(n^{3/4})$ [8]	$O(\sqrt{n}\log^{7/2} n \log\log n)$	$\Omega(\sqrt{n})$	6

Table 1: Quantum query bounds, with $n = \max(r, c)$ for matrices and n vertices for polygons. The polygon baseline [8] holds for the diameter of a general point set, and it is a bound on time as well as on queries. All classical randomized algorithms need $\Omega(n)$ queries on these problems.

Scope. All bounds count queries on a noiseless model. The algorithm behind the bounds of Section 5 exists through the general theorem of [7], and it is not efficient in gates as stated. The algorithm of Section 4 is an explicit composition of quantum searches. Section 8 states which parts are formalized in Lean. The other parts are claims with the proofs given here.

2 Preliminaries

Let M be totally monotone, and let $p(i)$ be the leftmost column in which row i attains its maximum.

Lemma 2.1. *The function p is nondecreasing.*

Proof. Let $i < i'$ and suppose $j' := p(i') < j := p(i)$. By the choice of j, $M[i, j'] < M[i, j]$, since $j' < j$ and j is the leftmost maximum of row i. Total monotonicity gives $M[i', j'] < M[i', j]$, a contradiction with j' being a maximum of row i'. □

Leftmost maxima matter here. The matrix with rows $(1, 1)$ and $(2, 0)$ is totally monotone, and taking column 2 for the first row would hide the maximum of the second row. So every maximum finding below runs on the pair of the value and the negated column index.

Quantum primitives. A maximum over N items costs at most $\kappa\sqrt{N}$ queries with bounded error, for a constant κ [5]. If item i has a known cost bound t_i, then a search over the items costs $\tilde{O}(\sqrt{\sum_i t_i^2})$ queries [3].

3 Lower bounds

Theorem 3.1. *For $h \in \{0, 1\}^n$ let $M_h[i, j] = -(i - j)^2$ for $i \neq j$ and $M_h[i, i] = h_i$. Then M_h satisfies the reverse quadrangle inequality, and its maximum is the OR of h. So computing the maximum needs $\Omega(\sqrt{n})$ quantum queries and $\Omega(n)$ classical randomized queries.*

Proof. It suffices to check adjacent 2×2 blocks. Let $D = M[i, j] + M[i + 1, j + 1] - M[i, j + 1] - M[i + 1, j]$. The matrix with $h = 0$ has $D = 2$. Raising a diagonal entry by $h_i \leq 1$ raises D by h_i on a block with that entry on its main diagonal and lowers D by h_i on a block with it on the other diagonal. A block contains at most two diagonal entries, and when it contains two, both lie on its main diagonal. So $D \geq 1$ in every case. □

4 An explicit nested search

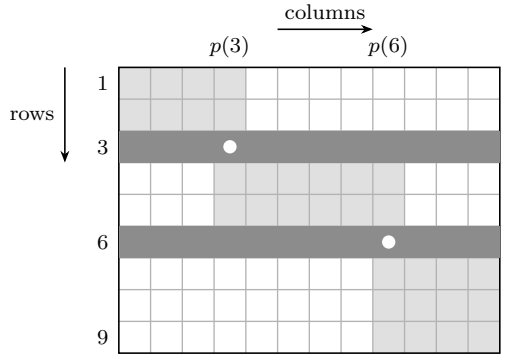


Figure 1: Depth one on a 9×12 matrix with $k = 2$ chosen rows. By Lemma 2.1, the maxima of the remaining rows lie in the shaded blocks, whose column widths sum to at most $c + k$.

At depth 0, search all rc cells by maximum finding, at $\kappa\sqrt{rc}$ queries. At depth d the algorithm is as follows.

Algorithm 1 NestedSearch(d) on rows $s, \dots, t$ and columns $l, \dots, u$

1: $r \leftarrow t - s + 1$, $k \leftarrow \lceil r^{1/(d+1)} \rceil$; choose k evenly spaced rows
2: compute the leftmost maxima of the chosen rows in bisection order: find p for the middle chosen row on the current window, then continue above it with the window $[l, p]$ and below it with the window $[p, u]$
3: this leaves $k + 1$ blocks of rows with windows of widths c_i
4: **return** the largest of the row maxima found and the results of NestedSearch($d - 1$) on the blocks, using a quantum search over the blocks

Cost of a level. At level j of the bisection the windows of the 2^j rows have total width at most $c + 2^j$. So that level costs at most $\kappa\sqrt{2^j(c + 2^j)}$, since $\sum_i \sqrt{x_i} \le \sqrt{m \sum_i x_i}$ for m terms. The sum over the levels is $O(\sqrt{k(c + k)})$, and the windows of the blocks satisfy $\sum_i c_i \le c + k$.

The overhead C. Let the search over the blocks cost $C\sqrt{\sum_i t_i^2}$ queries, where the t_i are known cost bounds. The factor C is polylogarithmic in n, and it includes three things beyond [2, 3]. Inner runs have bounded error and need error reduction and uncomputation at each level. All k row maxima must be right, which adds a factor $\log k$ to the first term. And a maximum over the blocks needs more than a search for one marked item, which the prefix bisection of Section 5 provides at polylogarithmic cost.

Theorem 4.1. *The cost at depth d is $T_d(r, c) \le O(C^d)\sqrt{r + c} \cdot r^{1/(2d+2)}$. So depth 1 gives $\tilde{O}(n^{3/4})$, and depth $d \approx \sqrt{\log n / \log C}$ gives $\sqrt{n} \cdot 2^{O(\sqrt{\log n \log\log n})}$ queries.*

Proof. Correctness follows from Lemma 2.1: each row has its leftmost maximum in its window, so the windows contain a largest entry.

For the cost, use induction with $a_d = 1/(2d+2)$, where $a_0 = 1/2$ matches $T_0 = \kappa\sqrt{rc}$. Up to constant factors,

$$T_d(r,c) \le \sqrt{k(c+k)} + C\sqrt{\sum_i T_{d-1}(r/k, c_i)^2} \le \sqrt{k(r+c)} + C^d\sqrt{r+c}\,(r/k)^{a_{d-1}}.$$

With $k = r^{1/(d+1)}$ the first term is $\sqrt{r+c}\,r^{1/(2d+2)}$, and the second has exponent $(1-\frac{1}{d+1})\cdot\frac{1}{2d} = \frac{1}{2d+2}$. □

At depth 1 there is no nesting: compute about $\sqrt{r}$ row maxima, and then run one maximum finding over the $O(n^{3/2})$ remaining cells.

5 Removing the overhead

Nesting pays C at each level. The filtered factorization norm has no such loss.

Definition 5.1 ([7])**.** Let D be a set of inputs, and let $\Delta_j[x,y] = 1$ if $x_j \ne y_j$ and 0 otherwise. For a matrix A on $D \times D$,

$$\gamma_2(A \mid \Delta) = \min\ \max\Big(\max_x \sum_j \|u_{xj}\|^2,\ \max_y \sum_j \|v_{yj}\|^2\Big) \quad \text{where} \quad A[x,y] = \sum_j \Delta_j[x,y]\langle u_{xj}, v_{yj}\rangle.$$

For f on D let $F_f[x,y] = 1$ if $f(x) = f(y)$ and 0 otherwise, and put $\gamma(f) = \gamma_2(J - F_f \mid \Delta)$, where J is the all-ones matrix.

By [7], the query complexity Q and the general adversary bound satisfy $Q(f) = \Theta(\mathrm{Adv}^{\pm}(f))$ and $\mathrm{Adv}^{\pm}(f) \le \gamma(f) \le 2\,\mathrm{Adv}^{\pm}(f)$. So $Q(f) = O(\gamma(f))$ and $\gamma(f) = O(Q(f))$ for every f.

Lemma 5.2 (Sequential composition)**.** *Let $A : D \to E$, and for each $a \in E$ let G_a be a function on $D_a = A^{-1}(a)$. Let $h(x) = G_{A(x)}(x)$. Then $\gamma(h) \le 2\gamma(A) + \max_a \gamma(G_a)$.*

Proof. Let $\alpha(x,y) = 1$ if $A(x) \ne A(y)$ and $\beta(x,y) = 1$ if $h(x) \ne h(y)$, with both 0 otherwise. Then

$$\beta = \alpha + (1-\alpha)\beta - (1-\beta)\alpha,$$

since the right side expands to $\alpha + \beta - \alpha\beta - \alpha + \alpha\beta$. The first term is $J - F_A$. The second term, on a pair with $A(x) = A(y) = a$, is $J - F_{G_a}$, and it vanishes on other pairs. Tagging vectors with $A(x)$, so that vectors of different classes are orthogonal, gives a factorization of cost $\max_a \gamma(G_a)$. The third term is $J - F_A$ restricted to pairs with $h(x) = h(y)$. Tagging the factorization of $J - F_A$ with $h(x)$ gives it at cost $\gamma(A)$. Filtered factorizations add by direct sum, and their costs add. □

Lemma 5.3 (OR)**.** *For binary functions $g_1, \dots, g_k$ on D and $\mathrm{OR}(x) = \bigvee_i g_i(x)$, we have $\gamma(\mathrm{OR}) \le \sqrt{\sum_i \gamma(g_i)^2}$.*

Proof. Let $a_i = \gamma(g_i)$ and $S = \sqrt{\sum_i a_i^2}$, and let (u^i, v^i) factor $J - F_{g_i}$ with squared norm at most a_i on each input. If some $a_i = 0$ then g_i is constant on D: the constant 1 makes OR constant, and the constant 0 can be dropped. So take all $a_i > 0$. If $\mathrm{OR}(x) = 0$, put

$$U_x = \bigoplus_i \sqrt{a_i}\,u^i_x, \qquad W_x = \bigoplus_i \sqrt{a_i}\,v^i_x,$$

both of squared norm at most $\sum_i a_i^2 = S^2$. If $\mathrm{OR}(y) = 1$, fix $i = i(y)$ with $g_i(y) = 1$, and put $T_y = a_i^{-1/2} u_y^i$ and $V_y = a_i^{-1/2} v_y^i$ on block i and 0 elsewhere, both of squared norm at most 1. For $\mathrm{OR}(x) = 0$ and $\mathrm{OR}(y) = 1$ the function $g_{i(y)}$ differs on x and y, so $\sum_j \Delta_j \langle U_{xj}, V_{yj} \rangle = 1$ and $\sum_j \Delta_j \langle T_{yj}, W_{xj} \rangle = 1$. Now use two orthogonal slots:

$$\begin{aligned} \mathrm{OR}(x) = 0: &\quad L_x = S^{-1/2}(U_x, 0), \quad R_x = S^{-1/2}(0, W_x), \\ \mathrm{OR}(y) = 1: &\quad L_y = S^{1/2}(0, T_y), \quad R_y = S^{1/2}(V_y, 0). \end{aligned}$$

Pairs with different OR give 1 in both orders. Pairs with the same OR give 0, since their nonzero slots differ. All squared norms are at most S. □

Theorem 5.4. *On totally monotone $r \times c$ matrices and for fixed θ, the function $F(M) = [\max M \geq \theta]$ has $\gamma(F) \leq (2\kappa + 1)\lceil \log_2(r+1) \rceil \sqrt{r+c}$. So $Q(F) = O(\sqrt{n} \log n)$.*

Proof. Fix rows $s, \ldots, t$ and columns $l, \ldots, u$, with $r = t - s + 1$ and $c = u - l + 1$. Let $D[s, t, l, u]$ be the totally monotone inputs whose leftmost maxima on these rows lie in $[l, u]$, and let $F[s, t, l, u]$ be the statement that some entry in these rows and columns is at least θ. On this domain that is the same as some row maximum being at least θ. Let $\Gamma(r, c)$ be the largest γ over such domains, with $\Gamma(0, c) = 0$.

For $r = 1$ this is an OR of c bits, and each bit has $\gamma = 1$, so $\Gamma(1, c) \leq \sqrt{c}$ by Lemma 5.3.

For the step let $m = \lfloor (s + t)/2 \rfloor$ and $A(x) = p(m)$. Maximum finding on $[l, u]$ computes A, so $\gamma(A) \leq \kappa\sqrt{c}$, where κ now also includes the constant of $\gamma = O(Q)$. On $A^{-1}(p)$, Lemma 2.1 gives the domain $D[s, m-1, l, p] \cap D[m+1, t, p, u]$, and on it

$$F[s, t, l, u] = [x_{m,p} \geq \theta] \vee F[s, m-1, l, p] \vee F[m+1, t, p, u].$$

Restriction to a subdomain does not raise γ. Lemmas 5.2 and 5.3 give, with $r_1 + r_2 = r - 1$ and $c_1 + c_2 = c + 1$,

$$\Gamma(r, c) \leq 2\kappa\sqrt{c} + \sqrt{1 + \Gamma(r_1, c_1)^2 + \Gamma(r_2, c_2)^2}.$$

Let $\ell(r) = \lceil \log_2(r+1) \rceil$ and $a = 2\kappa + 1$. Assume $\Gamma(r_i, c_i) \leq a\, \ell(r_i) \sqrt{r_i + c_i}$, where $\ell(r_i) \leq \ell(r) - 1$. Since $(r_1 + c_1) + (r_2 + c_2) = r + c$, the square root is at most $1 + a(\ell(r) - 1)\sqrt{r+c}$. So $\Gamma(r, c) \leq 2\kappa\sqrt{c} + 1 + a(\ell(r) - 1)\sqrt{r+c} \leq a\, \ell(r) \sqrt{r+c}$. □

Corollary 5.5. *For any range of the entries, the maximum of a totally monotone matrix and its position take $O(\sqrt{n} \log^3 n \log\log n)$ queries.*

Proof. Deleting rows keeps total monotonicity, so the proof of Theorem 5.4 applies to any subset of rows and to the strict test "some entry exceeds θ". Fix a random order of the rows, and let S be the rows with a maximum above the current θ. Bisect on prefixes of the order with the decision, with its error reduced by a factor $O(\log\log n)$. This gives the first row of S in the order, which is uniform in S, using $O(\log n)$ decisions. Read the maximum of that row by maximum finding at $O(\sqrt{c})$ queries, and set θ to it. In expectation S halves, so $O(\log r)$ rounds suffice. □

Remark 5.6. By [7], any factorization gives a query algorithm built from reflections, so the algorithm exists and uses $O(\gamma)$ queries, but no bound on time follows. The recursion could be run by subroutine composition [6] or transducers [4]. We have not worked out either form.

6 Diameter of a convex polygon

The input is an array with the vertices $v_0, \ldots, v_{n-1}$ of a strictly convex polygon in counterclockwise order, and a query returns one vertex. The task is to find the largest distance between two vertices.

Lemma 6.1. *Let $a_1, \ldots, a_s$ and $b_1, \ldots, b_t$ be two arcs of consecutive vertices, disjoint, in this order around the polygon. Then $M[i, j] = |a_i b_j|$ satisfies the reverse quadrangle inequality, and the matrix of squared distances is totally monotone.*

Proof. For $i < i'$ and $j < j'$, the segments $a_i b_j$ and $a_{i'} b_{j'}$ are the diagonals of a convex quadrilateral, so they cross at a point P. By the triangle inequality,

$$|a_i b_j| + |a_{i'} b_{j'}| = |a_i P| + |P b_j| + |a_{i'} P| + |P b_{j'}| \geq |a_i b_{j'}| + |a_{i'} b_j|.$$

Squares compare the same way as distances within a row, so the squared matrix is totally monotone, and it has integer entries on integer coordinates. □

Theorem 6.2. *Deciding whether the diameter is at least a given value takes $O(\sqrt{n} \log^{3/2} n)$ queries, and the diameter itself takes $O(\sqrt{n} \log^{7/2} n \log\log n)$ queries. At least $\Omega(\sqrt{n})$ quantum and $\Omega(n)$ classical randomized queries are necessary.*

Proof. Halve the sequence of vertices recursively. Every pair is separated at exactly one node of the recursion, where it appears in a matrix for two arcs of sizes $s + t = n/2^\ell$, and there are 2^ℓ such matrices at level ℓ. One matrix query is at most four vertex queries, with uncomputation, so by Theorem 5.4 and $\gamma = O(Q)$ each matrix has $\gamma \leq O(\log n)\sqrt{n/2^\ell}$ in the vertex model. Lemma 5.3 over all matrices gives the squared cost $O(\log^2 n) \sum_\ell 2^\ell \cdot n/2^\ell = O(n \log^3 n)$.

For the value, run the prefix bisection of Corollary 5.5 on the vertices. A prefix restricts each matrix to the pairs with an endpoint in the prefix, which is two totally monotone submatrices at the same cost bound. When the bisection returns a vertex, read its farthest vertex by maximum finding at $O(\sqrt{n})$ queries.

For the lower bounds, take $n/2$ antipodal pairs of rational points on a circle of radius R. These points are in strictly convex position, the antipodal pairs have distance $2R$, and no other pair reaches it. Take one vertex v from each pair and move it to $(1 + \varepsilon)v$ if its bit is 1. For small ε this keeps strict convexity, which is an open condition on finitely many points, and a moved vertex has distance $(2 + \varepsilon)R$ from its antipode. So the decision whether the diameter exceeds $2R$ computes the OR of $n/2$ bits. □

7 Simulation and circuits

The repository has simulation scripts, some of which sample each search from the exact outcome law of quantum search with a budget per search, and a circuit test.

- The nested search agrees with brute force on 1500 runs, and the polygon routine agrees with brute force and with rotating calipers on 144 runs.
- At depth 1 and $n = 2048$, the count is $35\, n^{3/4}$ to $51\, n^{3/4}$ oracle applications at a budget of 16 per search, with no wrong answers at budget 16 or more.
- The classical algorithm of [1] reads $3.5n$ to $5n$ entries in the same runs, so depth 1 passes it only for n of order 10^4, by extrapolation.
- With every search a circuit in Qiskit, on at most 8 qubits, 30 runs give no wrong answer.

- The recursion of Theorem 5.4, evaluated on single inputs with a base cost of one, gives 1.96 to 2.01 times $\sqrt{n}\log_2 n$ for n from 64 to 4096.

These runs test the constructions. The constant κ in Theorem 5.4 is not known, so the logarithmic factors there say nothing at practical sizes.

8 Formal verification

The Lean files in the repository build with no `sorry`, and `#print axioms` reports only the standard axioms.

Proved outright. Total monotonicity from the reverse quadrangle inequality, Lemma 2.1 and the windows, the hard family of Theorem 3.1, the classical skeleton of the algorithm of Section 4 with its width sums, Lemma 6.1, Lemmas 5.2 and 5.3, the recurrence of Theorem 5.4 on γ, the round count of Corollary 5.5, and the arithmetic of Theorem 4.1.

Stated as hypotheses. The two theorems of [7] quoted in Section 5, the cost of maximum finding, and the recursion for T_d in the arithmetic of Theorem 4.1. With these, Theorem 5.4 holds in Lean as a bound on Q, with a constant independent of r, c, the alphabet and θ.

Not formalized. The definitions of Q and of the adversary bound are not unfolded by any proof, so their match with the literature rests on reading them. Also not formalized are Corollary 5.5 as a bound on Q (error reduction, adaptive composition and the step from mean to worst case), the quantum algorithm and the variable-time search of Theorem 4.1, the OR lower bounds of Theorem 3.1, and Theorem 6.2 apart from Lemma 6.1.

9 Open questions

- An explicit algorithm with $\tilde{O}(\sqrt{n})$ queries and a matching bound on gates.
- The exact power of the logarithm. Between $\Omega(\sqrt{n})$ and $O(\sqrt{n}\log n)$ the decision problem is open.
- A dynamic program or geometric problem in which this query bound becomes a gain in time.

Use of AI

The proofs, the code and the Lean files were developed with Claude Opus 5.5 (Anthropic), which was also used to search the literature, to review the proofs and to draft the text.

References

[1] A. Aggarwal, M. Klawe, S. Moran, P. Shor, R. Wilber. Geometric applications of a matrix-searching algorithm. Algorithmica 2, 1987.

[2] A. Ambainis. Quantum search with variable times. STACS 2008, arXiv:quant-ph/0609168.

[3] A. Ambainis. Variable time amplitude amplification and quantum algorithms for linear algebra problems. STACS 2012, arXiv:1010.4458.

[4] A. Belovs, S. Jeffery, D. Yolcu. Taming quantum time complexity. arXiv:2311.15873.

[5] C. Dürr, P. Høyer. A quantum algorithm for finding the minimum. arXiv:quant-ph/9607014.

[6] S. Jeffery. Quantum subroutine composition. arXiv:2209.14146.

[7] T. Lee, R. Mittal, B. Reichardt, R. Špalek, M. Szegedy. Quantum query complexity of state conversion. FOCS 2011, arXiv:1011.3020.

[8] A. Zajakins. Quantum algorithms for computational geometry. Master's thesis, University of Latvia, 2023.